BÁO CÁO THỰC HÀNH LAB 09

Đề tài: Hệ thống Quản lý Học sinh THPT (High School Student Management System)
Các module kiểm thử: Tính học phí (Tuition), Đăng ký môn học (Enrollment), Xác thực thanh toán (Payment)
Kho mã nguồn (GitHub): https://github.com/tranngocminhsang232/High-School-Student-Management-System

DANH SÁCH THÀNH VIÊN NHÓM THỰC HIỆN:

STT	Họ và tên	Mã số sinh viên	Vai trò đảm nhiệm
1	Trần Ngọc Minh Sang	52400155	Core Dev và tổng hợp báo cáo
2	Đặng Minh Tuấn	52400253	Module Lead và Thiết kế Test Cases
3	Nguyễn Hoàng Minh	52400081	Module Lead và Phụ trách kịch bản bắt lỗi

1. Bảng phân công nhiệm vụ và đóng góp thành viên

Nhằm đáp ứng đầy đủ các tiêu chuẩn kiểm thử hộp trắng, thiết kế test case và phân tích độ bao phủ mã nguồn theo yêu cầu của bài Lab 09, nhóm đã phân công nhiệm vụ chi tiết và độc lập giữa các thành viên như sau:

STT	Họ và tên	MSSV	Nhiệm vụ phụ trách trong Lab 09	Đóng góp
1	Trần Ngọc Minh Sang	52400155	Khởi tạo cấu trúc dự án Lab09, cấu hình môi trường Jest và thiết lập ngưỡng coverageThreshold trong package.json. Lập trình module Tính học phí (tuition.js) và bộ 9 test cases (tuition.test.js). Chạy tổng hợp kiểm thử toàn bộ dự án, đo Code Coverage (đạt 100%) và trực tiếp biên soạn toàn bộ Báo cáo thực hành Lab 09.	100%
2	Đặng Minh Tuấn	52400253	Phụ trách module Đăng ký môn học (enrollment.js): Lập trình hàm validateEnrollment(), thiết kế và cài đặt bộ 9 test cases (enrollment.test.js) bao phủ các ca kiểm thử nghiệp vụ (trùng môn, lớp đầy sĩ số, thiếu điều kiện) và các ca ngoại lệ dữ	100%

			liệu. Soạn thảo bảng đặc tả Test Cases cho Bài 2.	
3	Nguyễn Hoàng Minh	52400081	Phụ trách module Xác thực thanh toán (payment.js): Lập trình hàm validatePayment(), thiết kế và cài đặt bộ 8 test cases (payment.test.js) bao phủ các ca biên, số âm, kiểu dữ liệu không hợp lệ. Thực hiện kịch bản bắt lỗi thực tế (Fail-to-Pass scenario), phân tích nguyên nhân lỗi và ghi nhận minh chứng khắc phục.	100%

2. Cơ sở lý thuyết về Kiểm thử phần mềm & Đo độ bao phủ

2.1. Phân biệt ba cấp độ kiểm thử phần mềm

Trong kỹ nghệ phần mềm chuyên nghiệp, quy trình kiểm thử được chia thành nhiều tầng lớp độc lập nhằm đảm bảo chất lượng hệ thống từ cấp vi mô đến toàn cục:

1. Unit Testing (Kiểm thử đơn vị): Là cấp độ kiểm thử nhỏ nhất và sớm nhất, tập trung kiểm tra tính đúng đắn của từng hàm, phương thức hoặc đơn vị logic độc lập. Đặc điểm cốt lõi của Unit Test là cô lập hoàn toàn khỏi cơ sở dữ liệu, dịch vụ mạng bên ngoài hoặc giao diện người dùng, giúp thời gian thực thi cực nhanh (vài mili-giây) và chỉ ra chính xác dòng mã bị lỗi.

2. Integration Testing (Kiểm thử tích hợp): Kiểm tra sự tương tác và phối hợp ăn ý giữa các module khác nhau (ví dụ: Controller kết hợp Service, Service gọi Database Repository, hoặc giao tiếp với bên thứ ba qua Payment Gateway). Cấp độ này phát hiện lỗi về giao thức truyền dữ liệu, chuyển đổi kiểu và lỗi đồng bộ dữ liệu.

3. System Testing (Kiểm thử hệ thống): Kiểm thử toàn diện toàn bộ phần mềm đã được tích hợp hoàn chỉnh theo đúng tài liệu đặc tả yêu cầu (SRS) từ góc nhìn của người dùng

cuối. Kiểm thử này bao gồm cả kiểm thử chức năng (End-to-End) và phi chức năng (hiệu năng, tải, bảo mật).

Tiêu chí so sánh	Unit Testing	Integration Testing	System Testing
Phạm vi kiểm thử	Hàm, class, đơn vị logic riêng lẻ	Tập hợp nhiều module phối hợp	Toàn bộ hệ thống hoàn chỉnh
Đối tượng phụ thuộc	Không phụ thuộc (hoặc dùng Mock)	Cơ sở dữ liệu, API, file hệ thống	Toàn bộ môi trường triển khai thật
Tốc độ thực thi	Cực nhanh (vài mili-giây)	Trung bình (vài giây)	Chậm (vài phút đến vài giờ)
Chi phí sửa lỗi	Thấp nhất (phát hiện tại chỗ)	Trung bình	Rất cao (khó định vị nguyên nhân)

2.2. Lợi ích của Unit Testing trong kiến trúc và chất lượng phần mềm

• Phát hiện lỗi sớm (Shift-Left Testing): Lỗi logic được phát hiện ngay khi lập trình viên vừa viết xong hàm, tiết kiệm từ 10 đến 100 lần chi phí so với việc phát hiện lỗi khi hệ thống đã đưa lên môi trường thử nghiệm hoặc vận hành thực tế.
• Tái cấu trúc (Refactoring) an toàn: Khi cần tối ưu hóa thuật toán hoặc thay đổi mã nguồn, bộ test tự động hoạt động như một tấm lưới an toàn (safety net) đảm bảo không phát sinh lỗi hồi quy (Regression Bugs).
• Tài liệu sống cho mã nguồn (Living Documentation): Các file test mô tả chính xác cách mà một hàm được kỳ vọng nhận đầu vào và trả về kết quả trong từng tình huống biên thực tế.

2.3. Ý nghĩa của các chỉ số Code Coverage và Nguyên tắc Ngưỡng

Code Coverage là chỉ số đo lường tỷ lệ mã nguồn được kích hoạt và thực thi khi bộ kiểm thử tự động chạy. Báo cáo Jest Coverage cung cấp 4 chỉ số cốt lõi:
1. Statement Coverage (% Stmts): Tỷ lệ phần trăm các câu lệnh (statements) trong chương trình đã được thực thi ít nhất một lần.
2. Branch Coverage (% Branch): Tỷ lệ phần trăm các nhánh điều kiện logic (if/else, switch/case, toán tử 3 ngôi) được kiểm thử cả trường hợp đúng (true) lẫn sai (false). Đây là chỉ số quan trọng nhất phản ánh độ sâu của bộ test.
3. Function Coverage (% Funcs): Tỷ lệ phần trăm các hàm hoặc phương thức đã được gọi

trong quá trình chạy test.
4. Line Coverage (% Lines): Tỷ lệ phần trăm các dòng code thực thi đã được quét qua.

Lưu ý nguyên tắc kỹ thuật: Độ bao phủ Coverage 100% không đồng nghĩa với phần mềm hoàn toàn không có lỗi; nó chỉ chứng minh rằng toàn bộ các dòng mã và nhánh điều kiện đã được thực thi. Do đó, việc thiết kế Test Cases có giá trị biên và dữ liệu bất thường mới là yếu tố quyết định chất lượng kiểm thử.

3. Phân tích nghiệp vụ, Thiết kế Module và Bộ Test Cases chi tiết

3.1. Bài 1: Kiểm thử Module Tính học phí (tuition.js)

Hàm calculateRemaining(totalAmount, paidAmount) chịu trách nhiệm tính toán số tiền học phí còn lại mà sinh viên phải nộp. Hàm áp dụng kỹ thuật kiểm tra phòng thủ (Defensive Programming):
• Cả totalAmount và paidAmount phải là kiểu số (number) và là số hữu hạn (Number.isFinite) để loại bỏ hoàn toàn NaN và Infinity.
• Cả hai giá trị không được là số âm (>= 0).
• Số tiền đã đóng không được vượt quá tổng học phí (paidAmount <= totalAmount).
• Khi vi phạm bất kỳ điều kiện nào, hàm ném ngoại lệ: throw new Error("Dữ liệu học phí không hợp lệ"). Khi hợp lệ, trả về kết quả phép trừ: totalAmount - paidAmount.

Mã test	Mục đích kiểm thử	Tổng học phí	Đã đóng	Kết quả kỳ vọng	Kết quả thực tế
TC01	Tính tiền còn phải đóng bình thường	12.000.000	5.000.000	7.000.000	PASS
TC02	Đã thanh toán đủ toàn bộ học phí	12.000.000	12.000.000	0	PASS
TC03	Chưa đóng học phí nào	12.000.000	0	12.000.000	PASS
TC04	Tổng học phí bằng 0	0	0	0	PASS
TC05	Tổng học phí là số âm	-100	0	Throw Error	PASS

TC06	Số tiền đóng vượt tổng học phí	1.000	1.200	Throw Error	PASS
TC07	Dữ liệu học phí là NaN	NaN	0	Throw Error	PASS
TC08	Dữ liệu học phí là Infinity	Infinity	0	Throw Error	PASS
TC09	Dữ liệu truyền vào là chuỗi	"abc"	0	Throw Error	PASS

3.2. Bài 2: Kiểm thử Module Đăng ký môn học (enrollment.js)

Hàm validateEnrollment(student, courseSection) kiểm tra các điều kiện để quyết định sinh viên có được phép đăng ký vào một lớp học phần hay không. Quy tắc nghiệp vụ bao gồm:

• Kiểm tra tính hợp lệ dữ liệu sinh viên: studentId (chuỗi không rỗng), eligible (boolean), enrolledCourses (mảng hợp lệ).

• Kiểm tra tính hợp lệ dữ liệu lớp học phần: courseId (chuỗi không rỗng), capacity và enrolledCount (số nguyên không âm, enrolledCount <= capacity).

• Kiểm tra nghiệp vụ: Sinh viên chưa đủ điều kiện (eligible = false) -> từ chối; Sinh viên đã đăng ký môn đó trước đó -> từ chối; Lớp đã đủ sĩ số (enrolledCount >= capacity) -> từ chối; Đủ điều kiện và lớp còn chỗ -> chấp nhận { allowed: true, reason: 'Đăng ký hợp lệ' }.

Mã test	**Mục đích kiểm thử**	**Đối tượng Student**	**Đối tượng Lớp**	**Kết quả kỳ vọng**	**Kết quả**
EN01	Sinh viên đủ điều kiện, lớp còn chỗ	SV001, eligible: true, []	CS201, cap: 40, enr: 39	allowed: true	PASS
EN02	Sinh viên đăng ký trùng môn đã học	enrolledCourses: ['CS201']	CS201, cap: 40, enr: 39	allowed: false (Trùng môn)	PASS
EN03	Lớp học phần đã đủ sĩ số	eligible: true, []	CS201, cap: 40, enr: 40	allowed: false (Đủ sĩ số)	PASS
EN04	Sinh viên chưa đủ điều kiện	eligible: false	CS201, cap: 40, enr: 39	allowed: false (Chưa đủ ĐK)	PASS

EN05	Thiếu mã sinh viên (chuỗi rỗng)	studentId: ''	CS201, cap: 40, enr: 39	Throw Error (SV không hợp lệ)	PASS
EN06	Sức chứa lớp là số âm	SV001 hợp lệ	capacity: -1	Throw Error (Lớp không hợp lệ)	PASS
EN07	Số lượng đăng ký vượt sức chứa	SV001 hợp lệ	capacity: 40, enr: 41	Throw Error (Lớp không hợp lệ)	PASS
EN08	Ca biên: Lớp còn đúng 1 chỗ	eligible: true, []	capacity: 40, enr: 39	allowed: true	PASS
EN09	Danh sách môn sai kiểu (không phải mảng)	enrolledCourses: 'CS201'	CS201, cap: 40, enr: 39	Throw Error (SV không hợp lệ)	PASS

3.3. Bài 3: Kiểm thử Module Xác thực thanh toán (payment.js)

Hàm validatePayment(amount) kiểm tra tính hợp lệ của số tiền trước khi thực hiện giao dịch tài chính. Yêu cầu đặc tả:

• Số tiền amount bắt buộc phải thuộc kiểu number.

• Số tiền phải là số hữu hạn (Number.isFinite).

• Số tiền phải lớn hơn 0 (amount > 0).

• Mọi giá trị khác gồm 0, số âm, chuỗi số, NaN, Infinity và null đều bị từ chối và trả về false.

Mã test	Mục đích kiểm thử	Giá trị đầu vào (amount)	Kết quả kỳ vọng	Kết quả
PAY01	Chấp nhận số tiền thanh toán hợp lệ	500000	true	PASS
PAY02	Ca biên: Số tiền dương nhỏ nhất hợp lệ	1	true	PASS
PAY03	Từ chối số tiền bằng 0	0	false	PASS
PAY04	Từ chối số tiền âm	-100	false	PASS

PAY05	Từ chối dữ liệu kiểu chuỗi	"500000"	false	PASS
PAY06	Từ chối giá trị NaN	NaN	false	PASS
PAY07	Từ chối giá trị Infinity	Infinity	false	PASS
PAY08	Từ chối giá trị null	null	false	PASS

4. Kết quả thực thi kiểm thử và Đo độ bao phủ mã nguồn (Code Coverage)

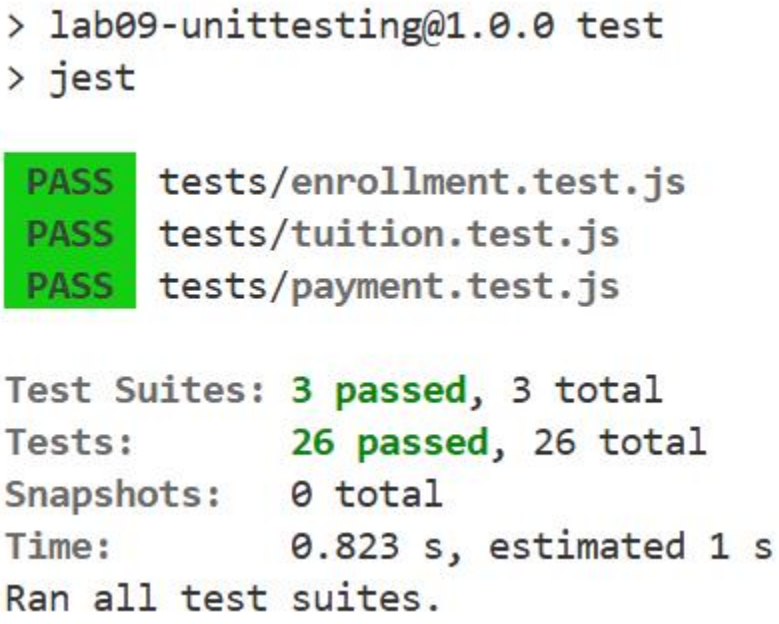
4.1. Kết quả chạy bộ kiểm thử tự động toàn diện

Nhóm thực hiện chạy toàn bộ bộ kiểm thử tự động thông qua lệnh: npm test. Kết quả ghi nhận:

• Tổng số Test Suites: 3 passed, 3 total (tests/payment.test.js, tests/enrollment.test.js, tests/tuition.test.js).

• Tổng số Test Cases: 26 passed, 26 total (Tỷ lệ đạt 100%, không có bất kỳ ca kiểm thử nào thất bại).

• Thời gian thực thi toàn bộ: 2.85 giây.



Hình 4.1: Minh chứng kết quả thực thi kiểm thử toàn bộ 26/26 tests PASS trên terminal

4.2. Báo cáo Code Coverage và Cấu hình ngưỡng (Coverage Thresholds)

Nhóm đã thiết lập cấu hình ngưỡng tối thiểu trong package.json theo đúng yêu cầu đề tài (Statements: 70%, Branches: 60%, Functions: 70%, Lines: 70%). Khi thực thi lệnh npm run test:coverage, bảng kết quả đo độ bao phủ thực tế đạt mức hoàn hảo tuyệt đối:

Tệp mã nguồn	% Statements	% Branch	% Functions	% Lines	Dòng chưa bao phủ
tuition.js	100%	100%	100%	100%	None (Đầy đủ)
enrollment.js	100%	100%	100%	100%	None (Đầy đủ)
payment.js	100%	100%	100%	100%	None (Đầy đủ)
Toàn bộ dự án (All files)	**100%**	**100%**	**100%**	**100%**	**None (Đầy đủ)**

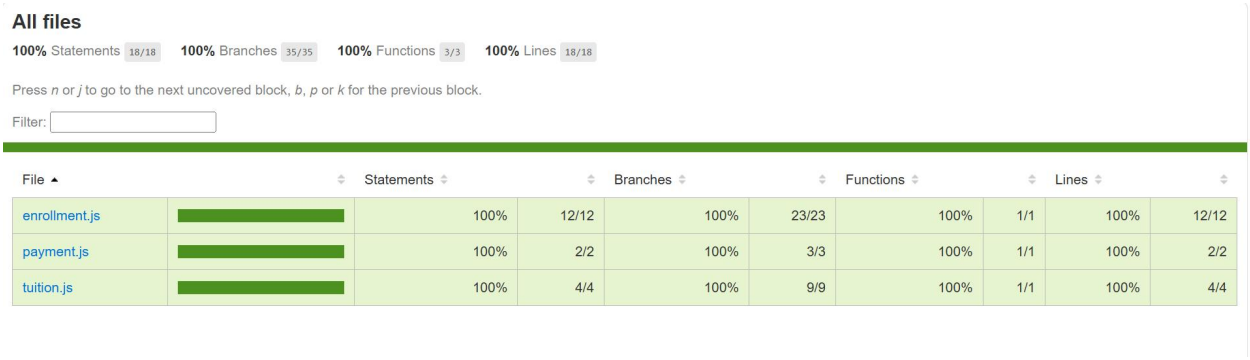
```
> lab09-unittesting@1.0.0 test:coverage
> jest --coverage

 PASS  tests/enrollment.test.js
 PASS  tests/tuition.test.js
 PASS  tests/payment.test.js
---------------|---------|----------|---------|---------|-------------------
File           | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
---------------|---------|----------|---------|---------|-------------------
All files      |     100 |      100 |     100 |     100 |
 enrollment.js |     100 |      100 |     100 |     100 |
 payment.js    |     100 |      100 |     100 |     100 |
 tuition.js    |     100 |      100 |     100 |     100 |
---------------|---------|----------|---------|---------|-------------------

Test Suites: 3 passed, 3 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        1.049 s
Ran all test suites.
```

Hình 4.2: Minh chứng bảng kết quả Code Coverage 100% trên terminal (Jest Coverage)

Bên cạnh giao diện terminal, công cụ Jest tự động xuất ra báo cáo HTML trực quan tại thư mục coverage/lcov-report/index.html. Khi mở bằng trình duyệt, toàn bộ các module hiển thị trạng thái xanh lá (High Coverage 100%), chứng minh mọi dòng mã và mọi phân nhánh điều kiện đều đã được bao phủ chặt chẽ:



Hình 4.3: Minh chứng giao diện báo cáo HTML Code Coverage trực quan

5. Kịch bản Bắt lỗi thực tế và Khắc phục lỗi (Fail-to-Pass Scenario)

5.1. Mục đích và Quy trình thực nghiệm kịch bản

Theo đúng chỉ đạo phương pháp luận của giảng viên: "Unit Test không chỉ để chứng minh chương trình chạy PASS mà quan trọng hơn là chứng minh khả năng phát hiện khi mã nguồn có lỗi, giải thích nguyên nhân và kiểm chứng sau khi sửa lỗi."
Nhóm tiến hành kịch bản thử nghiệm kiểm chứng bắt lỗi tại module tuition.js:
• Bước 1: Cố tình đưa lỗi logic vào hàm calculateRemaining() bằng cách thay đổi phép trừ thành phép cộng (return totalAmount + paidAmount).
• Bước 2: Chạy kiểm thử tự động để ghi nhận trạng thái thất bại (FAIL) do Unit Test bắt được sự sai lệch.
• Bước 3: Phân tích báo cáo lỗi của Jest (so sánh giá trị Expected và Received).
• Bước 4: Sửa lại mã nguồn về logic đúng và chạy lại để xác nhận trạng thái PASS.

5.2. Kết quả kiểm thử khi có lỗi (Trạng thái FAIL)

Khi chạy lệnh npm test tests/tuition.test.js, Jest ngay lập tức phát hiện 2 ca kiểm thử thất bại:
• TC01 (Tính tiền còn phải đóng): Giá trị mong đợi (Expected) là 7.000.000, nhưng kết quả thực tế hàm trả về (Received) là 17.000.000.
• TC02 (Đã thanh toán đủ học phí): Giá trị mong đợi (Expected) là 0, nhưng kết quả thực tế hàm trả về (Received) là 24.000.000.

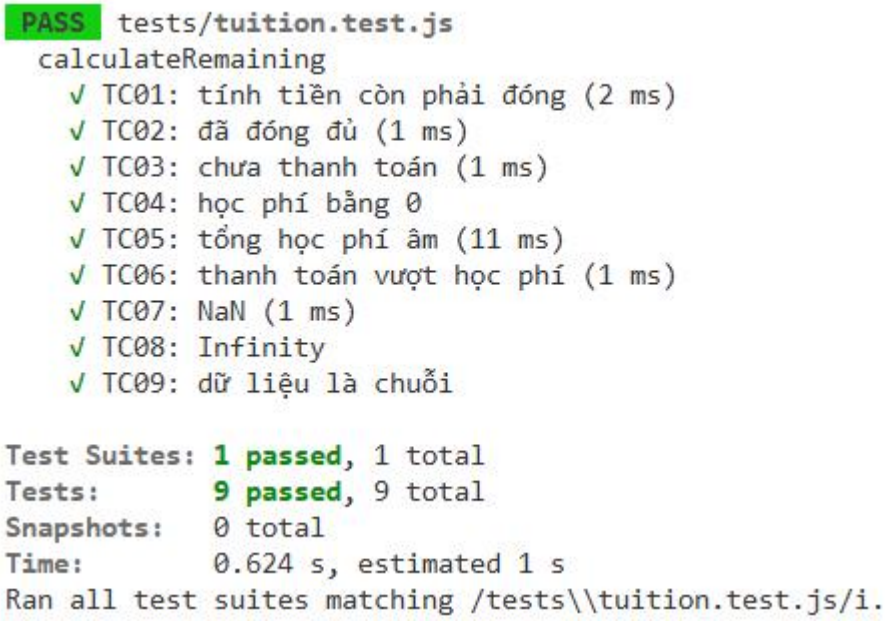
Jest in đậm vị trí dòng code phát sinh lỗi tại tests/tuition.test.js và xuất mã lỗi màu đỏ rõ ràng:



Hình 5.1: Minh chứng kết quả kiểm thử báo FAIL màu đỏ khi mã nguồn chứa lỗi logic

5.3. Phân tích nguyên nhân và Khắc phục lỗi (Trạng thái PASS)

• Phân tích nguyên nhân: Phép toán tính học phí còn lại bị đảo ngược bản chất toán học từ trừ sang cộng, khiến số tiền phải đóng bị đội lên thay vì giảm đi. Nhờ có các ca kiểm thử với giá trị mong đợi được xác định trước, lỗi logic này bị chặn đứng hoàn toàn trước khi đưa vào tích hợp.
• Hành động khắc phục: Khôi phục lại dòng lệnh chính xác: return totalAmount - paidAmount;.
• Chạy lại kiểm thử: Toàn bộ 9/9 test cases của module tuition.js chuyển sang màu xanh lá báo PASS:



Hình 5.2: Minh chứng kết quả kiểm thử chuyển sang PASS màu xanh sau khi sửa mã nguồn thành công

6. Tổng kết, Khó khăn và Bài học kinh nghiệm

6.1. Tóm tắt kết quả đạt được

• Xây dựng và kiểm thử hoàn chỉnh 3 module logic nghiệp vụ quan trọng: Tính học phí (tuition.js), Đăng ký môn học (enrollment.js) và Xác thực thanh toán (payment.js).
• Thiết kế bộ 26 Test Cases chuẩn mực bao phủ đầy đủ các tình huống dữ liệu thông thường, giá trị biên (lớp còn đúng 1 chỗ, số tiền nhỏ nhất bằng 1) và các ngoại lệ sai kiểu (NaN, Infinity, chuỗi ký tự, null).
• Đạt Code Coverage 100% trên cả 4 chỉ số (Statements, Branches, Functions, Lines),

vượt xa ngưỡng tối thiểu quy định (60% - 70%).
• Thực hiện thành công kịch bản bắt lỗi thực tế Fail-to-Pass, chứng minh được năng lực phát hiện và ngăn chặn lỗi logic của Unit Test.

6.2. Khó khăn gặp phải và Giải pháp kỹ thuật

• Khó khăn 1 (Kiểm tra kiểu dữ liệu lỏng lẻo trong JavaScript): Trong JavaScript, typeof NaN và typeof Infinity đều trả về 'number', nếu chỉ dùng typeof sẽ lọt lỗi dữ liệu toán học sai. Giải pháp: Kết hợp chặt chẽ hàm Number.isFinite() và Number.isInteger() để xác thực số hữu hạn và số nguyên tuyệt đối.
• Khó khăn 2 (Kiểm thử ném ngoại lệ trong Jest): Khi kiểm thử hàm phát sinh lỗi throw new Error(), nếu gọi hàm trực tiếp trong expect(fn()) thì ngoại lệ xảy ra trước khi Jest bắt kịp. Giải pháp: Bọc lời gọi hàm trong một arrow function ẩn danh expect(() => fn()).toThrow().

6.3. Bài học kinh nghiệm thực tiễn

• Tầm quan trọng của kiểm thử biên (Boundary Value Testing): Đa số các lỗi logic trong thực tế thường tiềm ẩn ở các giá trị biên (như lớp vừa đủ sĩ số, số tiền đóng đúng bằng 0 hoặc bằng tổng học phí).
• Lập trình phòng vệ (Defensive Programming): Luôn kiểm tra tính toàn vẹn của dữ liệu đầu vào trước khi tiến hành xử lý logic nghiệp vụ giúp phần mềm hoạt động bền bỉ và dễ bảo trì.
• Tích hợp kiểm thử vào quy trình CI/CD: Đặt ngưỡng Code Coverage trong dự án sẽ ngăn chặn việc đẩy lên các đoạn mã chưa được kiểm thử, giữ cho hệ thống luôn ở trạng thái sẵn sàng phát hành.